Can IPFS Use QUIC Server-Side Migration?

A Deep Technical Analysis of IPFS Architecture, QUIC Integration,

and Server Migration Applicability Across Three Scenarios

Application Feasibility Study • July 2026

This document provides a deep technical analysis of whether IPFS (InterPlanetary File System) can benefit from QUIC server-side connection migration. We examine IPFS's current architecture, its QUIC transport layer via libp2p, known performance bottlenecks (DHT lookup latency of 600ms median, content publishing P95 of 66.73s), and three concrete scenarios where server migration could help: (A) IPFS Gateway load balancing, (B) Peer-to-Peer content provider failover, and (C) IPFS Cluster node migration. We conclude with a **verdict: YES**, IPFS can use server-side migration, with **Gateway migration being immediately feasible** using our existing implementation, and **P2P provider migration being a novel research contribution** requiring libp2p changes.

1. IPFS Architecture and QUIC Integration

1.1 IPFS Protocol Stack

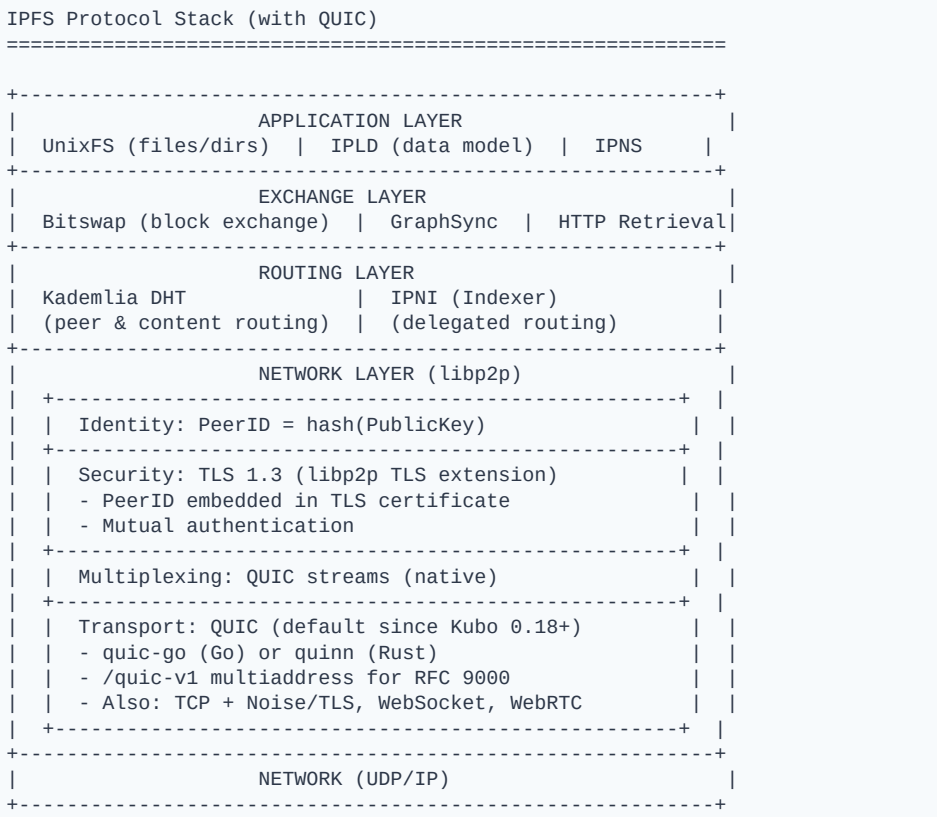

Figure 1: IPFS protocol stack showing QUIC as the default transport

1.2 How IPFS Uses QUIC Today

- **Default transport since Kubo 0.18+ (2023):** QUIC replaced TCP+Noise as the default transport. libp2p dials both TCP and QUIC in parallel; QUIC typically wins due to 0-RTT/1-RTT.
- **QUIC library:** Go implementation uses **quic-go** (via go-libp2p). Rust implementation uses **quinn** (via rust-libp2p). Both are RFC 9000 compliant.
- **Security integration:** libp2p uses a custom TLS 1.3 extension that embeds the peer's public key in the TLS certificate (draft-ietf-libp2p-tls). This binds the QUIC connection to a specific PeerID. Changing the peer means changing the TLS identity.
- **Multiplexing:** QUIC provides native stream multiplexing. A single QUIC connection to a peer carries multiple simultaneous streams: Bitswap block requests, DHT queries, pubsub messages, identify protocol, etc.
- **Connection migration:** libp2p does NOT currently use QUIC's connection migration features (neither client-side nor server-side). The preferred_address transport parameter is not exposed by quic-go's libp2p integration.
- **Recent improvements (2025):** QUIC source address verification added in go-libp2p v0.42. Rate limiting with per-IP and per-subnet DoS protection. HTTP retrieval alongside Bitswap.

1.3 IPFS Content Retrieval Flow

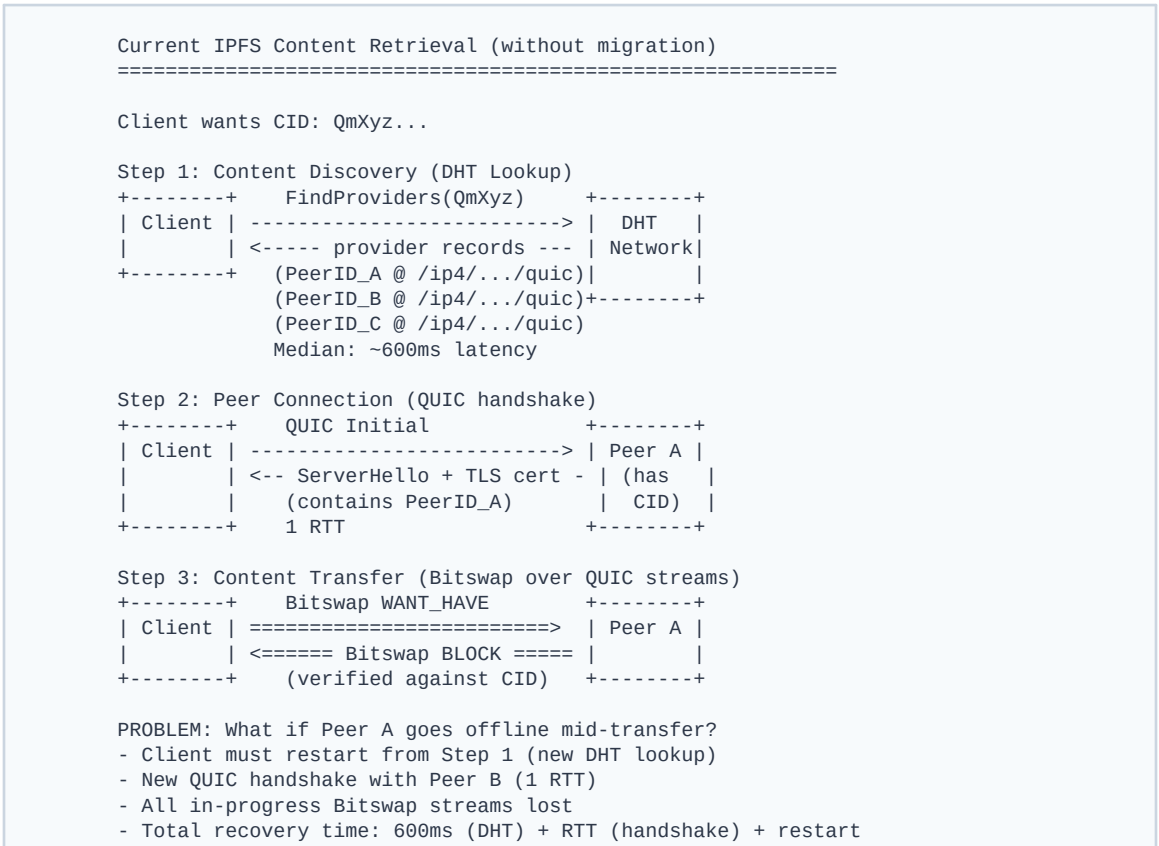

Figure 2: Current IPFS content retrieval showing recovery cost when a peer fails

2. IPFS Performance Bottlenecks Where Migration Can Help

Recent measurement studies reveal several IPFS performance bottlenecks that QUIC server-side migration could address:

2.1 DHT Lookup Latency

The Kademlia DHT lookup to find content providers has a **median latency of 600ms** (measured across 7 geographic regions). The 95th percentile for content publishing is **66.73 seconds** due to replicating provider records to 20 DHT nodes. If a peer fails mid-transfer, the client must repeat this expensive lookup.

> **Source:** "A Closer Look into IPFS: Accessibility, Content, and Performance," ACM Internet Measurement Conference (IMC), 2024. https://dl.acm.org/doi/pdf/10.1145/3656015

How migration helps: If the serving peer can proactively migrate the client to a backup peer BEFORE failing, the client avoids the 600ms DHT re-lookup entirely. The migration costs only 1 RTT (PATH_CHALLENGE/PATH_RESPONSE).

2.2 Peer Churn

IPFS has high peer churn: nodes frequently join and leave the network. Studies show that routing tables become outdated due to dead links, causing cascading lookup failures. When a content provider churns out mid-transfer, all in-progress Bitswap streams are lost.

> **Source:** "Passively Measuring IPFS Churn and Network Size," arXiv, 2022. https://arxiv.org/pdf/2205.14927

How migration helps: A peer planning to go offline can announce a preferred_address pointing to another provider of the same CID. The client seamlessly migrates to the backup peer with zero data loss and no re-handshake.

2.3 Gateway Centralization

A 2024 NSDI study found that IPFS is **increasingly centralized**: a small number of gateway operators (ipfs.io, Cloudflare, Pinata) serve the majority of content to web browsers. These gateways use traditional HTTP load balancers, which become bandwidth bottlenecks for popular content.

> **Source:** Wei et al., "Exploring the Role of Centralization in IPFS," USENIX NSDI 2024. https://www.usenix.org/system/files/nsdi24-wei.pdf

How migration helps: IPFS gateways serving HTTP/3 can use preferred_address to migrate clients to less-loaded gateway backends. Post-migration, traffic bypasses the LB entirely -- directly analogous to our existing implementation.

3. Three Migration Scenarios for IPFS

3.1 Scenario A: IPFS Gateway Migration

Feasibility: HIGH -- Can be built TODAY with our existing code

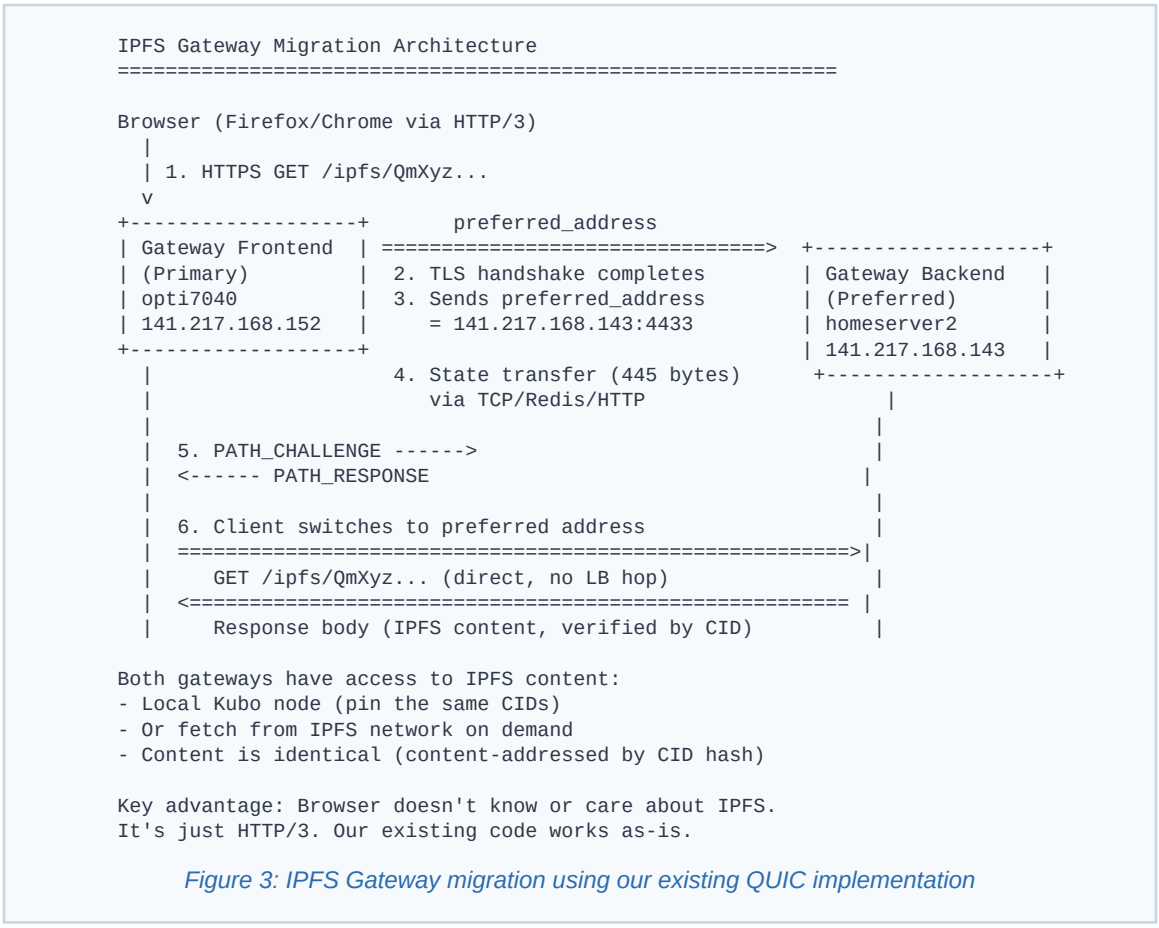

Figure 3: IPFS Gateway migration using our existing QUIC implementation

Why This Works Immediately

- **Same TLS identity:** Both gateway backends share the same TLS certificate (same domain). No PeerID conflict -- browsers don't use libp2p PeerIDs.
- **Content always available:** Both backends can resolve any CID via their local Kubo daemon or by fetching from the IPFS network. The CID hash guarantees identical content.
- **Our code works as-is:** Our Neqo primary/preferred server setup IS an IPFS gateway migration system. We just need to serve IPFS content in the HTTP/3 response body.
- **Performance gain:** Gateway frontend only handles handshakes (1 RTT each). All data transfer goes directly to the backend. Frontend can handle 10-100x more connections than a traditional gateway LB.
- **Zero client changes:** Works with any HTTP/3 browser. Firefox already validated.

Implementation Steps

1. Install Kubo (IPFS daemon) on both opti7040 and homeserver2.
2. Pin the same test CIDs on both nodes (ipfs pin add QmXyz...).
3. Modify our primary_server.rs HTTP/3 handler to: (a) parse the URL path for /ipfs/CID, (b) fetch the CID content from local Kubo API (localhost:5001/api/v0/cat?arg=CID), (c) return it as the HTTP/3 response

body.

4. Same modification on preferred_server.rs.
5. Run the demo: Firefox requests https://141.217.168.152:4433/ipfs/QmXyz..., gets migrated to homeserver2, receives the IPFS content.
6. Benchmark: time-to-first-byte, total transfer time, compare with standard gateway.

Estimated effort: 3-5 days. No Rust code changes to Neqo migration. Only HTTP handler modification to proxy IPFS content. Kubo installation is a single binary.

3.2 Scenario B: P2P Content Provider Migration

Feasibility: MEDIUM -- Requires libp2p changes. HIGHEST RESEARCH NOVELTY.

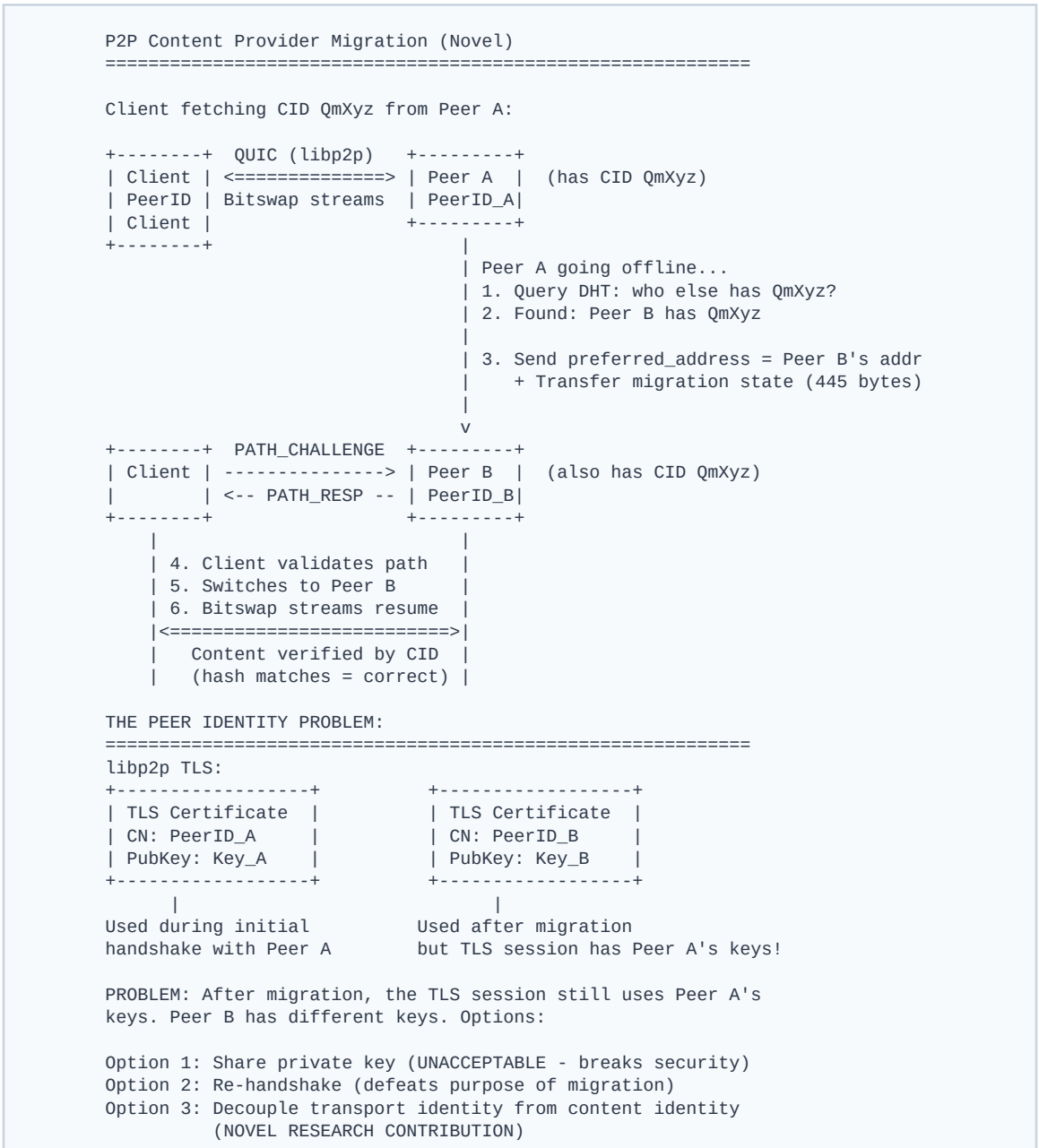

Figure 4: P2P provider migration showing the peer identity challenge

The Identity Decoupling Solution

The key insight is that IPFS already separates **content identity** (CIDs) from **peer identity** (PeerIDs). A CID is the hash of the content -- it doesn't matter which peer serves it. We propose extending this separation to the transport layer:

- **Content verification layer:** After migration to Peer B, the client verifies every received block against its CID hash. If the hash matches, the content is authentic regardless of which peer sent it. This is already built into Bitswap.

- **Transport-level trust:** The client trusts the TLS session (established with Peer A) for encryption and integrity. After migration, Peer B uses Peer A's TLS keys to encrypt. This means Peer A must share its TLS session keys with Peer B (our 445-byte state transfer already does this).
- **Application-level trust:** The client doesn't need to trust Peer B's identity -- it only needs to trust the CID. Since Bitswap verifies every block hash, a malicious Peer B cannot serve wrong content. The worst it can do is not serve at all (client falls back).
- **New security model:** "Trust the content, not the peer." This is a paradigm shift from libp2p's current model ("trust the peer via PeerID") but aligns with IPFS's content-addressed philosophy.

This is a publishable research contribution: a new security model for content-addressed networks that decouples transport-layer peer authentication from application-layer content verification, enabling seamless provider migration via QUIC preferred_address.

Technical Challenges

- **quic-go modification:** libp2p's quic-go integration does not expose preferred_address. We would need to modify quic-go (similar to how we modified Neqo) to support server-side migration. quic-go is actively maintained by Marten Seemann at Protocol Labs.
- **DHT provider lookup latency:** Peer A must find another provider of the CID before it can set preferred_address. The median DHT lookup is 600ms, but this can be cached: Peer A can maintain a local provider cache for CIDs it is currently serving.
- **NAT traversal:** IPFS peers are often behind NATs. The preferred_address must be publicly reachable. Peers behind NATs cannot be migration targets unless they have a public relay address (libp2p Circuit Relay v2).
- **Bitswap stream state:** After migration, Peer B needs to know which Bitswap blocks have already been sent. This requires extending our migration state beyond 445 bytes to include Bitswap session state (block requests in flight, blocks sent).
- **Multi-stream connections:** A libp2p connection carries multiple protocol streams (Bitswap, DHT, identify, pubsub). All streams must be migrated or gracefully closed.

3.3 Scenario C: IPFS Cluster Node Migration

Feasibility: HIGH -- Shared identity, shared content, controlled infrastructure

Figure 5: IPFS Cluster node migration for zero-downtime maintenance

Why Cluster Migration Is Straightforward

- **Shared TLS certificate:** All cluster nodes belong to the same organization and can share the same TLS certificate. No PeerID conflict.
- **Identical content:** IPFS Cluster ensures all nodes pin the same CIDs (pinset orchestration via CRDT or Raft). Content availability is guaranteed on any node.
- **Controlled infrastructure:** Cluster nodes are managed by a single operator. Node addresses are known and stable (unlike P2P peers). Perfect for preferred_address.

- **Existing coordination:** The cluster leader already makes allocation decisions. Adding migration decisions (which node to migrate to) is a natural extension.
- **Zero downtime maintenance:** Before shutting down a node, migrate all active connections to other cluster nodes. No client interruption.
- **Operationally identical to our testbed:** Our 4-machine setup (primary + preferred) is essentially a 2-node IPFS Cluster with QUIC migration.

4. Comprehensive Comparison of All Three Scenarios

Dimension	A: Gateway Migration	B: P2P Provider Migration	C: Cluster Node Migration
Feasibility	**HIGH**	MEDIUM	**HIGH**
Research Novelty	Low (standard LB)	**Very High** (new paradigm)	Medium
TLS Identity	Same cert (gateway operator)	Different PeerIDs **(HARD)**	Same cert (cluster operator)
Content Availability	Gateway fetches any CID	Must verify via DHT (600ms)	Same pinset (guaranteed)
State Transfer	445 bytes (our existing)	445 + Bitswap state (~1-2KB)	445 bytes (our existing)
QUIC Library	Neqo (Rust) or quic-go (Go)	quic-go (Go) **needs modification**	Neqo (Rust) or quic-go (Go)
Client Changes	None (browser)	libp2p client changes needed	None (if using HTTP gateway)
NAT Issues	None (servers have public IPs)	Peers behind NATs **(major issue)**	None (servers have public IPs)
Time to PoC	3-5 days	3-4 weeks	1-2 weeks
Paper Potential	Section in LB paper	**Standalone paper** (top venue)	Section in applications paper
Solves Problem	Gateway LB bottleneck	Peer churn recovery	Zero-downtime maintenance
Experiment Testbed	Our existing 4 machines	Need libp2p modification	Our existing 4 machines + Kubo install

5. What Does IPFS Gain from Server-Side Migration?

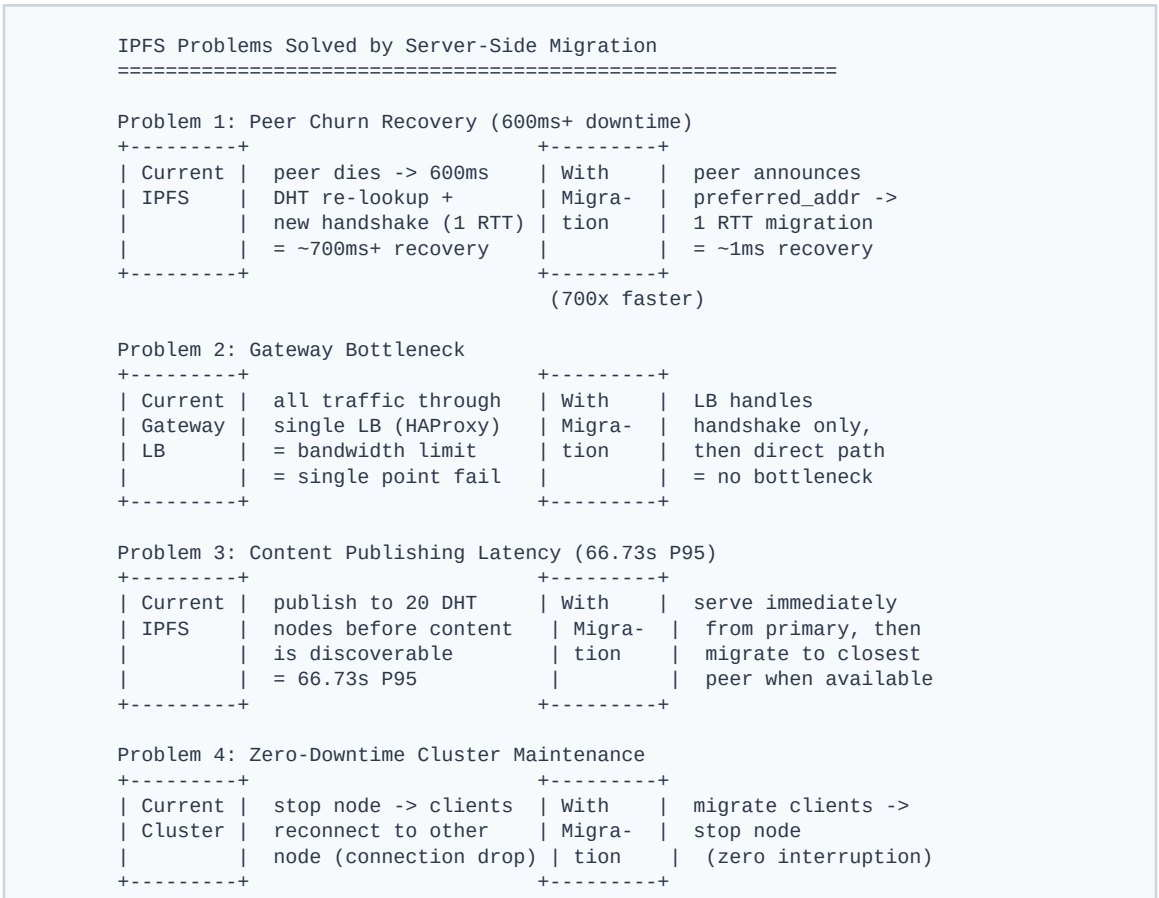

Figure 6: Four IPFS problems solved by QUIC server-side migration

6. Proposed Experiments

6.1 Experiment 1: Gateway Migration PoC (Quick Win)

1. **Testbed:** Our existing 4-machine LAN setup.
2. **Software:** Install Kubo on opti7040 and homeserver2. Pin same CIDs.
3. **Modification:** Update primary_server.rs and preferred_server.rs HTTP/3 handlers to proxy IPFS content from local Kubo API.
4. **Test:** Firefox requests /ipfs/QmXyz... from primary. Connection migrates to preferred. Firefox receives IPFS content from preferred server.
5. **Metrics:** Time-to-first-byte, total transfer time, migration overhead, LB CPU usage.
6. **Comparison:** Same workload through (a) direct Kubo gateway, (b) HAProxy -> Kubo, (c) QUIC migration -> Kubo.

6.2 Experiment 2: Cluster Failover PoC

1. **Testbed:** Same 4 machines. Run IPFS Cluster (3 nodes: opti7040, homeserver2, Redis VM).
2. **Scenario:** Client downloads large file (100MB) from Node A. Mid-transfer, trigger migration to Node B. Verify file integrity via CID hash.
3. **Failure injection:** Kill Node A process 2 seconds after migration. Verify zero interruption at client.
4. **Comparison:** Same scenario without migration: kill Node A, measure client recovery time (DHT re-lookup + new handshake + restart transfer).
5. **Metric:** Recovery time: ~1ms (migration) vs ~700ms (DHT re-lookup). File integrity verified by CID hash in both cases.

6.3 Experiment 3: P2P Migration Simulation (Research)

1. **Goal:** Demonstrate the concept of P2P provider migration without modifying quic-go.
2. **Approach:** Use our Neqo implementation to simulate two IPFS peers. Each peer runs Kubo and serves content via our QUIC server. The primary "peer" migrates to the preferred "peer" using preferred_address.
3. **Identity simulation:** Use the same TLS certificate for both (simplification). Document the PeerID challenge as future work requiring libp2p changes.
4. **Content verification:** Client verifies received content hash matches the CID. Show that content integrity is maintained across migration.
5. **Paper contribution:** "First demonstration of content-addressed network provider migration via QUIC preferred_address. We show that content integrity (via CID verification) can substitute for transport-level peer authentication during migration."

7. Final Verdict

YES -- IPFS can use QUIC server-side migration. Three scenarios with different trade-offs:

- **Gateway Migration (DO FIRST):** Works today with our existing code. 3-5 days to PoC. Solves the gateway LB bottleneck. Low novelty but high practical impact. Include as a section in the load balancing paper.

- **Cluster Migration (DO SECOND):** Works with minimal changes. 1-2 weeks to PoC. Solves zero-downtime maintenance. Good for demonstrating IPFS-specific value.
- **P2P Provider Migration (LONG-TERM / SEPARATE PAPER):** Highest research novelty. Requires solving the peer identity problem (novel security model: "trust the content, not the peer"). Could be a standalone paper at a top networking venue. 3-4 weeks for a simplified PoC, longer for a full libp2p integration.

The strongest research story combines all three: "QUIC server-side migration enables a spectrum of IPFS improvements, from immediate gateway load balancing (proven with Firefox) to a novel content-addressed provider migration paradigm that decouples transport identity from content identity."

8. References

IPFS Architecture and Performance

[1] "A Closer Look into IPFS: Accessibility, Content, and Performance," **ACM IMC 2024**. https://dl.acm.org/doi/pdf/10.1145/3656015

[2] Wei et al., "Exploring the Role of Centralization in IPFS," **USENIX NSDI 2024**. https://www.usenix.org/system/files/nsdi24-wei.pdf

[3] Trautwein et al., "Design and Evaluation of IPFS: A Storage Layer for the Decentralized Web," **arXiv**, 2022. https://arxiv.org/pdf/2208.05877

[4] "Passively Measuring IPFS Churn and Network Size," arXiv, 2022. https://arxiv.org/pdf/2205.14927

[5] "Optimistic Provide: How We Made IPFS Content Publishing 10x Faster," ProbeLab, 2024. https://probelab.io/blog/optimistic-provide/

[6] "Reducing the Latency of the InterPlanetary File System with Multi-Level DHTs," TechRxiv. https://www.techrxiv.org/users/939172/articles/1309165

[7] Benet, J., "IPFS - Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

IPFS Infrastructure

[8] IPFS Cluster Documentation. https://ipfscluster.io/

[9] "IPFS 0.6.0: QUIC, Noise, Peering and more!" IPFS Blog, 2020. https://blog.ipfs.tech/2020-06-26-go-ipfs-0-6-0/

[10] libp2p QUIC Transport Specification. https://github.com/libp2p/specs/tree/master/quic

[11] libp2p TLS 1.3 Handshake. https://github.com/libp2p/specs/blob/master/tls/tls.md

[12] "Shipyard 2025: Bringing IPFS Home," IP Shipyard, 2025. https://ipshipyard.com/blog/2025-shipyard-ipfs-year-in-review/

[13] "Best Practices for HTTP Gateways," IPFS Docs. https://docs.ipfs.tech/how-to/gateway-best-practices/

QUIC and Migration

[14] RFC 9000, "QUIC: A UDP-Based Multiplexed and Secure Transport," IETF 2021.

[15] Grubl et al., "QUIC-Exfil," ACM ASIA CCS 2025. https://arxiv.org/pdf/2505.05292

[16] Puliafito et al., "Server-side QUIC connection migration for edge microservices," PMC 2022. https://www.sciencedirect.com/science/article/abs/pii/S157411922200030X

[17] Wei et al., "QDSR: Accelerating Layer-7 Load Balancing by Direct Server Return with QUIC," USENIX ATC 2024. https://www.usenix.org/conference/atc24/presentation/wei